

MeetingHQ Local Run and Deployment Runbook

This guide is a practical runbook for:

- running the app locally for testing
- deploying it to a Linux VM with Docker + Nginx
- deploying it to a managed hosting platform such as GCP and related options

It is written to match the repository's real startup contract and operational defaults from the current codebase.

1. What the app needs to run

MeetingHQ is a full-stack application with:

- Frontend: React + Vite
- Backend: FastAPI
- Database: PostgreSQL
- Cache / coordination: Redis
- Optional email delivery: SMTP provider for transactional mail

Runtime variables to configure

Use the repo's environment files as the source of truth:

- .env.example
- .env

Key values required for a clean start:

```
```.env
MEETINGHQ_ENVIRONMENT=local
MEETINGHQ_API_HOST=0.0.0.0
MEETINGHQ_API_PORT=8000
MEETINGHQ_DATABASE_URL=postgresql+asyncpg://meetinghq:meetinghq_dev_only@localhost:5432/meetinghq
MEETINGHQ_REDIS_URL=redis://localhost:6379/0
MEETINGHQ_JWT_SECRET=replace-with-at-least-32-random-characters
MEETINGHQ_API_CORS_ORIGINS=["http://localhost:5173","http://127.0.0.1:5173"]

INITIAL_SUPER_ADMIN_EMAIL=textabi12@gmail.com
INITIAL_SUPER_ADMIN_PASSWORD=ChangeMe123!
INITIAL_SUPER_ADMIN_FIRST_NAME=Abiodun
INITIAL_ORGANIZATION_NAME=MeetingHQ
INITIAL_WORKSPACE_NAME=Main Workspace
```

...

Notes:

- In Docker Compose, the DB and Redis hostnames are `postgres` and `redis`.
- In native local development on Windows or Linux host, change them to `localhost`.
- `VITE\_API\_URL` must be public-safe, because Vite embeds it into browser assets.

## 2. Recommended local test path (Docker Compose)

This is the easiest and best-supported local test path.

### ***Step 1: Copy the environment template***

```
```bash
cd meetinghq
cp .env.example .env
```
```

### ***Step 2: Strongly review the first-run bootstrap values***

Set these before the first start if you want the API to bootstrap the organization, workspace, and super admin on a fresh empty database:

```
```dotenv
INITIAL_SUPER_ADMIN_EMAIL=...
INITIAL_SUPER_ADMIN_PASSWORD=...
INITIAL_SUPER_ADMIN_FIRST_NAME=...
INITIAL_ORGANIZATION_NAME=MeetingHQ
INITIAL_WORKSPACE_NAME=Main Workspace
```
```

### ***Step 3: Start the stack***

```
```bash
docker compose up --build
```
```

This brings up:

- PostgreSQL container
- Redis container
- API container

- Web container

### ***Step 4: Verify the app***

Open:

- Web UI: `http://localhost:5173`
- API health: `http://localhost:8000/api/v1/health`
- API docs: `http://localhost:8000/api/docs`

Expected health response:

```
```json
{"success":true,"message":"Request completed successfully.","data":{"status":"ok"},"meta":{}}
```
```

### ***Step 5: Log in as the seed admin***

Use the bootstrap admin email and password from ``.env``.

## **3. Native local test path (Python + Node + host services)**

Use this when you want to run the API and web server directly on the host instead of inside Docker.

### ***Step 1: Prepare Python and Node tooling***

```
```bash
cd meetinghq
python -m venv .venv
source .venv/bin/activate
python -m pip install -e "apps/api[dev]"
```
```

On PowerShell, activate with:

```
```powershell
.\.venv\Scripts\Activate.ps1
```
```

### ***Step 2: Install frontend dependencies***

```
```bash
cd apps/web
```

```
npm install
```

```
...
```

Step 3: Make sure PostgreSQL and Redis are running on the host

- PostgreSQL should be reachable at `localhost:5432`

- Redis should be reachable at `localhost:6379`

If you are using the local `.env` in native mode, the hostnames should be `localhost` rather than `postgres` and `redis`.

Step 4: Start the backend

From the repo root:

```
```bash
```

```
uvicorn meetinghq_api.main:app --reload
```

```
```
```

Step 5: Start the web app

```
```bash
```

```
cd apps/web
```

```
npm run dev
```

```
```
```

Step 6: Run validation checks

Backend checks:

```
```bash
```

```
ruff check apps/api
```

```
ruff format --check apps/api
```

```
mypy --config-file apps/api/pyproject.toml
```

```
pytest apps/api
```

```
```
```

Frontend checks:

```
```bash
```

```
npm run lint
```

```
npm run format:check
```

```
npm run test
```

```
npm run build
```

```
npx playwright install chromium
npm run test:e2e
...
```

## 4. Linux VM deployment with Docker + Nginx

This is the most straightforward production-like deployment path.

### ***Step 1: Provision a Linux VM***

Use a VM such as:

- Ubuntu 22.04 LTS or 24.04 LTS
- Docker Engine
- Docker Compose
- Nginx
- A managed PostgreSQL service or local PostgreSQL service
- A managed Redis service or local Redis service

### ***Step 2: Clone the repo and copy the env file***

```
``bash
git clone
cd meetinghq
cp .env.example .env
...
```

### ***Step 3: Set production-safe variables***

Update `.env` with real deployment values:

```
``dotenv
MEETINGHQ_ENVIRONMENT=production
MEETINGHQ_DATABASE_URL=postgresql+asyncpg://:@:5432/
MEETINGHQ_REDIS_URL=redis://:6379/0
MEETINGHQ_JWT_SECRET=
MEETINGHQ_API_CORS_ORIGINS=["https://app.example.com"]
MEETINGHQ_SECURE_COOKIES=true
VITE_API_URL=https://api.example.com/api/v1
...
```

Do not commit real secrets into source control.

## ***Step 4: Use the production server URLs in the web app***

The frontend's API base URL should be the public origin of the API service.

## ***Step 5: Launch the stack***

```
```bash
docker compose up --build -d
```
```

If using the gateway profile:

```
```bash
docker compose --profile gateway up --build -d
```
```

## ***Step 6: Configure Nginx as the front door***

Use the repository's nginx reference config in [infrastructure/nginx/nginx.conf](infrastructure/nginx/nginx.conf) or your own reverse proxy configuration.

Typical map:

- `https://app.example.com` → frontend container
- `https://api.example.com` → backend API

## ***Step 7: Verify production health***

Check:

```
```bash
curl https://api.example.com/api/v1/health
curl https://api.example.com/api/v1/ready
```
```

Expected health state:

- `GET /api/v1/health` → liveness check
- `GET /api/v1/ready` → checks PostgreSQL and Redis readiness

# **5. Deployment to managed hosting platforms**

## ***5.1 GCP deployment options***

A good real-world GCP pattern is:

- Cloud Run for the FastAPI service
- Cloud SQL for PostgreSQL
- Memorystore for Redis
- Cloud Load Balancer or Cloud CDN for public entry
- Secret Manager for runtime secrets
- Artifact Registry or Cloud Build for image delivery

Recommended service split:

- Backend API on Cloud Run
- Frontend web shell on Cloud Run or a static host behind a CDN
- Postgres managed in Cloud SQL
- Redis managed in Memorystore

#### Minimum env wiring for GCP

```
```.env
MEETINGHQ_DATABASE_URL=postgresql+asyncpg://:@/
MEETINGHQ_REDIS_URL=redis://:6379/0
MEETINGHQ_API_CORS_ORIGINS=["https://"]
MEETINGHQ_JWT_SECRET=
MEETINGHQ_SECURE_COOKIES=true
VITE_API_URL=https:///api/v1
```
```

## ***5.2 Other hosting options***

Common alternatives include:

- Railway
- Render
- Azure App Service
- Fly.io
- DigitalOcean App Platform
- AWS ECS / EKS / Elastic Beanstalk

The deployment pattern is the same:

1. Put Postgres and Redis behind managed services.
2. Store secrets in the hosting platform secret manager.
3. Build and deploy the API container and web bundle.
4. Put HTTPS and a public domain in front of the endpoint.

## 6. Production deployment checklist

Before calling the deployment production-ready, confirm:

- PostgreSQL is reachable from the deployed API service
- Redis is reachable from the deployed API service
- JWT secret is stable and strong
- CORS only allows your real browser origins
- `MEETINGHQ\_SECURE\_COOKIES=true`
- SMTP is configured for invitations, password reset, and reminders
- Only the empty-install bootstrap variables are used during the first startup
- `.env` is never checked into source control for real deployments

## 7. Recommended command summary

### ***Local Compose***

```
```bash
docker compose up --build
```
```

### ***Local API***

```
```bash
uvicorn meetinghq_api.main:app --reload
```
```

### ***Local frontend***

```
```bash
cd apps/web
npm install
npm run dev
```
```

### ***Linux VM deployment***

```
```bash
docker compose up --build -d
```
```



## 8. Evidence from this workspace

This runbook is aligned to the repo's existing local and runtime contract:

- [README.md](README.md)
- [docs/local-development.md](docs/local-development.md)
- [docs/environment.md](docs/environment.md)
- [docker-compose.yml](docker-compose.yml)
- [infrastructure/nginx/nginx.conf](infrastructure/nginx/nginx.conf)

It also reflects the verified local runtime state you already confirmed: Redis can be reached with a live `PING` response and the API health endpoint returns `status: "ok"`.